GPUNTOBIT



The Book of *Triad*

Learn the language and the code of the triad2.0 project, from the first programs to the machine-code seed

I Foundations · II Triad-Core · III Triad-Full · IV Inside the project · V In the field

```
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
```

First edition · Language standard 0.2.0 · 2026

github.com/GPuntobit/triad2.0

Preface

Triad is a standalone programming language: block syntax and declared types, multiple dispatch, integrated logic programming, code as data. Its definition is a normative specification independent of any implementation, accompanied by an executable specification and a conformance suite with verifiable golden outputs. This book is the study path.

The book arises from a simple need: learning Triad does not mean only reading the specification, but building a mental model of the language, its constructs, and its runtime. The specification is normative and precise, but it is written to be implemented, not to be read like a novel. This volume turns it into a path: explanations, examples, exercises, profile markers. The explanations, examples, and exercises build, step by step, the mental model of the language. The specification remains the normative reference source, not a reading prerequisite.

The ideal reader already knows how to program in another language: knows what a variable, a function, a loop, and recursion are. This book does not teach programming in general, it teaches Triad. The language's influences (Julia, Prolog, Lisp) are documented in Appendix A of the specification: they are informative notes, not definitions. Triad is not a dialect of any of the three.

The volume gathers two complementary and independent texts. The *Triad 2.0 Manual* explains the language, proposes exercises, and closes with grammar, quick reference cards, an error table, a glossary, and appendices. The *Triad 2.0 Tutorial* is divided into five parts: first steps, Triad-Core, Triad-Full, project code, and reference. You can read the

Manual in full without the Tutorial, and the Tutorial in full without the Manual: each explains what it needs. The index that follows collects the chapters of both texts, so they can be found in a single place.

If you want to write executable programs today, start with the Tutorial, parts I and II; use the Manual to deepen the concepts and do the exercises. To study the complete standard, continue with part III of the Tutorial and the corresponding chapters of the Manual. Part IV is for those who want to explore or modify the repository.

The Manual's appendices guide you among the language's influences and the project's sources. The full normative texts are not reproduced in the volume: those who want the formal detail find them in the `SPEC.md` and `SPEC-CORE.md` files of the repository, which remain the definitive sources in case of divergence. The volume nonetheless contains the rules needed to understand and use the language: the reference to the specification is a deepening, not a dependency.

Two conformance profiles run through the whole book. Triad-Core is the bootstrap subset: values, functions, closures, control flow, lists, strings, I/O. Triad-Full is the complete standard. Every Core program is also a valid Full program and must behave the same way. The markers **[Core]** and **[Full]** indicate in which profile each construct exists; code blocks are marked `triad` for Core code and `triad-full` for code defined by the Full standard, not yet executable by the current runtime. Core examples can be tried with the reference JavaScript interpreter (`node seed/dev/triad_core.cjs`); fragments indicated as seed-specific are an exception.

The S0 seed is a Triad-Core interpreter written directly in x86-64 machine code, without a host language; today the executable seed is the one for Windows. It must not be confused with the JavaScript interpreter used to run Core examples and verify conformance. The bootstrap plan

provides S1 (a Triad interpreter written in Triad-Core), S2 (native compiler), S3 (self-hosting), and S4 (cross-platform parity). Where the runtime does not cover a construct, the text says so explicitly.

Happy reading, and good code.

— GPuntobit

Table of Contents

Triad 2.0 Manual

- How to read this manual
  - Who it is for
  - Two profiles, one language
  - Notation conventions
  - How to run the examples

Part I — The language

- 1. Lexicon
  - 1.1 Identifiers
  - 1.2 Numbers
  - 1.3 Strings
  - 1.4 Symbols
  - 1.5 Comments
  - 1.6 Separators and line endings
  - 1.7 The colon and question mark rule
  - 1.8 Keywords
  - 1.9 Advanced identifiers and conventions
  - 1.10 Escape and advanced strings
  - 1.11 Lexicon exercises
- 2. Values and types
  - 2.1 Types
  - 2.2 Type hierarchy
  - 2.3 Struct
  - 2.4 Conversions
  - 2.5 Type assertion
  - 2.6 The declared constraint
  - 2.7 Conditions accept only Bool

2.8 How a value is printed
3. Variables and assignment
4. Operators and precedence
4.1 Integer arithmetic
4.2 Logic and comparisons
5. Control flow
5.1 if / elseif / else [Core]
5.2 while, break, continue [Core]
5.3 let and begin [Core]
5.4 for and ranges [Full]
6.2 return, recursion, arity
6.3 Higher-order functions
6.4 The effect annotation !io
7. Types and multiple dispatch
7.1 The return type
7.2 Local functions
8. Lambda, closures and scope
8.1 Lambda [Core]
8.2 Closures
8.3 Scope rules
9. Lists
10. Strings, symbols and conversions
11. Errors as data
11.1 The Err form
11.2 Producing and propagating
11.3 Error categories
11.4 In Core: errors as tagged lists
12. Contracts
13. Budget
14. Traces
Glossary of trace events (Byrd ports)

15. Declared effects

16. Logic

16.1 Facts, rules, terms

22.1 Higher-order functions written in Core

22.2 Records as tagged lists

22.3 Environment as an association list

22.4 Shared mutable state

22.5 Building strings

22.6 Style rules

Part I — First steps

1. What Triad is

1.1 Two profiles: Core and Full

1.2 The bootstrap chain

2. Installing and running

2.1 Requirements

2.2 Running a file

2.3 Materializing the seed

2.4 Launchers bin/triad and bin/triad.cmd

2.5 Exit codes

2.6 The development CLI in brief

3. The first program

3.1 Command-line arguments

3.2 The recommended work cycle

Part II — Triad-Core step by step (runnable today)

4. Lexicon

5. Values, types and printing

5.1 Printing rules for print/println

5.2 Equality

6. Variables and assignment

7. Operators and expressions

7.1 Precedence (from lowest to highest)

7.2 Integer arithmetic

7.3 Logic and conditions

8. Control flow

8.1 if / elseif / else

8.2 while, break, continue

8.3 let and begin

9. Functions

9.1 Three ways to define a function

9.2 return, recursion, arity

9.3 Effect annotation !io

10. Lambda, closures and scope

10.1 Lambda

10.2 Closures

10.3 Scope rules

11. Lists

12. Strings, symbols and conversions

Part III — Triad-Full (the complete standard)

18. Lexicon and types Full (SPEC §1, §4)

19. Typed functions and multiple dispatch (SPEC §5)

20. Struct (SPEC §4)

21. Full control flow: for, range, comprehension, try (SPEC §2)

22. Logic: facts, rules, solve (SPEC §6)

23. Destructuring (SPEC §2)

24. Errors as data (SPEC §8)

25. Contracts (SPEC §9)

26. Budget (SPEC §10)

27. Traces (SPEC §11)

28. Declared effects (SPEC §12)

29. Code as data, uniform tree and macros (SPEC §3, §7)

29.1 The uniform tree

29.2 Quote, unquote, eval
29.3 Macro
29.4 Code that generates code
30. Builtin Full (SPEC §13)
31. Command line and REPL (SPEC §14)
32. Full examples with golden output
32.1 assaggio.td
32.2 famiglia.td
32.3 garanzie.td
32.4 macro.td
32.5 Contracts and budget together
13. Bootstrap primitives
14. Runtime errors
14.1 Errors as values, in Core
15. JS interpreter and Windows seed: differences

Part IV — The project code

33. Repository map
34. The reference JS interpreter (seed/dev/triad_core.cjs)
34.1 Architecture
34.2 Example: adding a builtin
34.3 Example: adding syntax
35. The S0 Windows seed and its generator
35.1 What it is
35.2 Internal conventions
35.3 The generator seed/dev/gen_seed.js
35.4 Flow to modify the seed
36. The Linux seed (E0)
37. Self-hosting: S1 in selfhost/
37.1 How S1 code is written today
37.2 Where S1 goes
38. Conformance: reading, writing and verifying cases

38.1 Format
38.2 The runner conformance/run.mjs
38.3 Adding a Core case
39. The development CLI triad-cli
39.1 Usage
39.2 Code structure
39.3 Adding a command
39.4 Standalone executable
40. AI benchmark (ai/)
41. Editor, launcher and site
42. Continuous integration
43. Contribution flow

Part V — Reference

44. Triad-Core grammar (EBNF)
45. Quick reference cards Core and Full
46. Debug and common problems
47. Glossary
48. State, known limits and historical fixes
49. FAQ

16. Idiomatic Triad-Core patterns

16.1 Higher-order functions written in Core

16.2 Records as tagged lists

16.3 Environment as an association list

16.4 Shared mutable state

16.5 Building strings

16.6 Style

17. Guided exercises

Exercise 1 — Powers and sums

Exercise 2 — Prime numbers

Exercise 3 — Reverse a list

Exercise 4 — Insertion sort

Exercise 5 — Word counter

Exercise 6 — A mini-tokenizer

Exercise 7 — Evaluating an expression

Part II — Exercises

23. Guided exercises (Triad-Core)

Exercise 1 — Powers and sums

Exercise 2 — Prime numbers

Exercise 3 — Reverse a list

Exercise 4 — Insertion sort

Exercise 5 — Word counter

Exercise 6 — A mini lexical analyzer

Exercise 7 — Evaluating an expression

Exercise 8 — A dictionary environment

24. Exercises on Triad-Full

Exercise 9 — Dispatch on struct

Exercise 10 — A knowledge base
Exercise 11 — Contracts in practice
Exercise 12 — A macro that measures
Part III — Reference
25. Grammar
25.1 Triad-Core (normative EBNF)
25.2 Triad-Full (normative EBNF)
26. Quick reference cards
26.1 Values
26.2 Control flow
26.3 Functions
27. Error table
27.1 In Core
27.2 In Triad-Full
28. Glossary
29. What runs today
Appendix A — Declared influences
Appendix B — Where to look

Triad 2.0 Tutorial

What you learn and how it is organized
How to use this tutorial
16.2 Goals
16.3 solve is a value
16.4 A predicate used as a function
16.5 Examples
17. Code as data
17.1 Quote, unquote, eval

18. Macros
18.1 Code that generates code
19. Builtin
20. Command line and REPL
20.1 The REPL
21. Bootstrap primitives
22. Style and idiomatic patterns
5.5 Comprehensions [Full]
5.6 try / catch [Full]
6. Functions
6.1 Three forms of definition [Core]